

Bonus Exercise 2 Report

Algorithms for Uncertainty Quantification (IN2345), SS26

Aleksandra Topalova (03745233), Alexandra Savova (03731070), Julian Kraus (03734087)
Group 36

1 Assignment 1

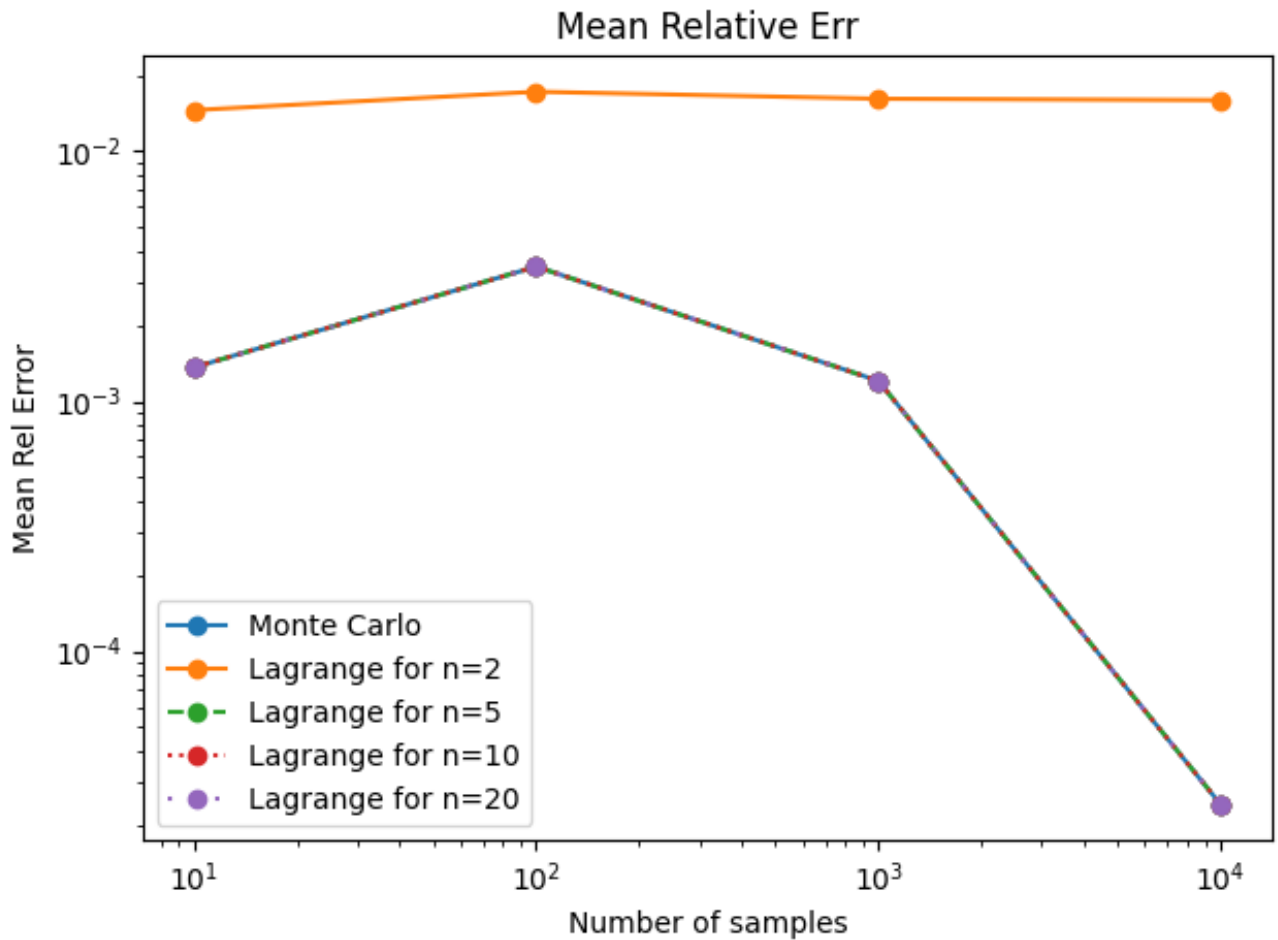


Figure 1: Relative error of mean

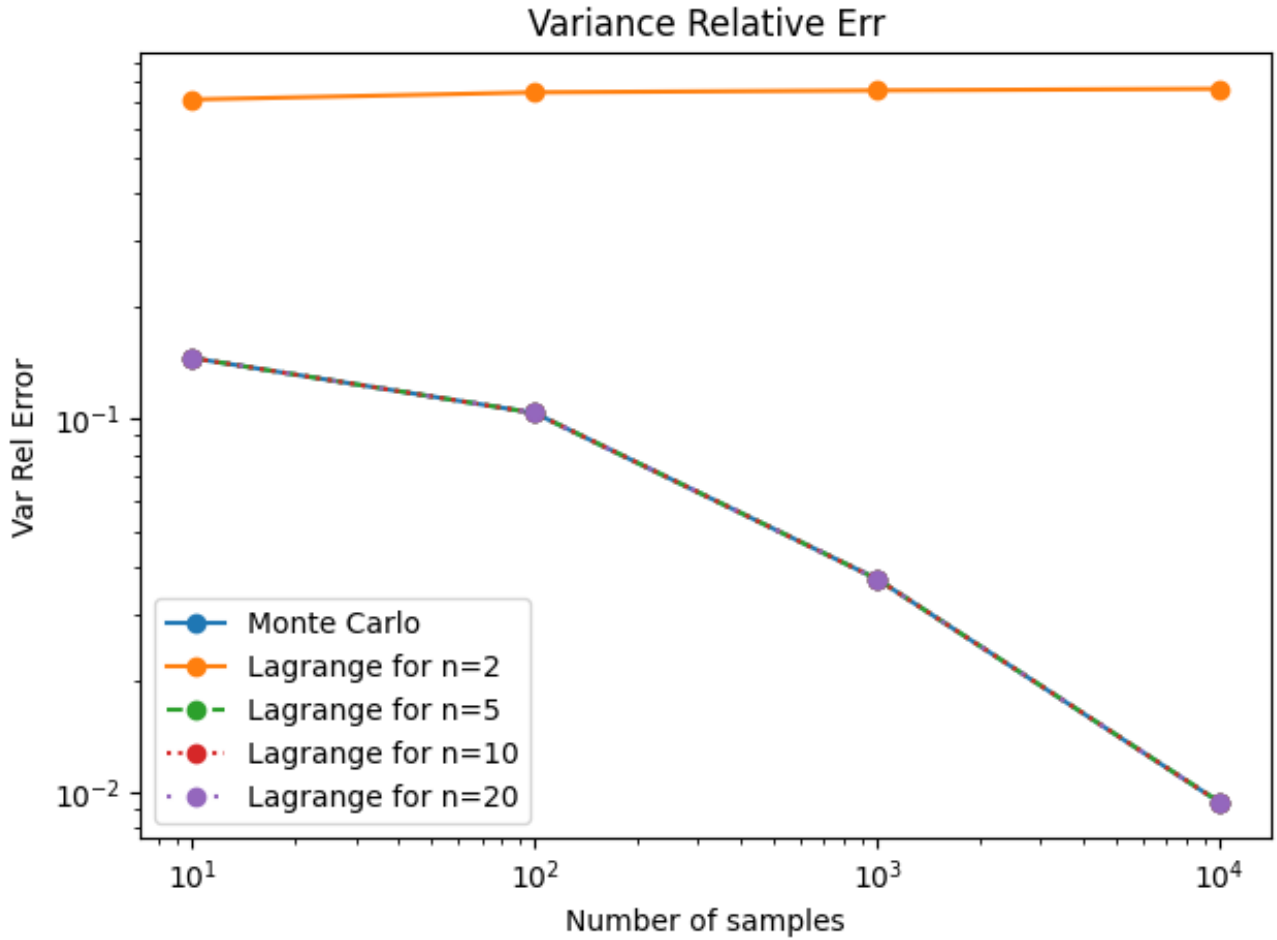


Figure 2: Relative errors of variance

1.1 Implementation

For the implementation of this assignment, first, we complete the four helper functions in *helpers.py*:

- For the generation of an interpolation grid, we reuse the code from *Exercise 5*. Specifically, the helper function generates two types of grids: uniform via the *NumPy* library and Chebyshev via the *SciPy* library (with consecutive scaling to the required interval $[low, high]$). The bonus exercise sheet hints at a formula for the computation of the Chebyshev grid points for an arbitrary interval $[low, high]$, but using *SciPy* with scaling produces equally useful results.
- For computing the relative errors, we reuse our implementation from *Bonus Exercise 1*. Notably, the helper function uses an unbiased variance estimator by setting $ddof = 1$ in the *NumPy* variance function call.
- For loading the reference values from a file, we also reuse our implementation from *Bonus Exercise 1*.
- Our oscillator simulation implementation is an adaptation of our code from *Bonus Exercise 1*. The code varies insignificantly in this exercise because the helper function receives the oscillator's ω parameter as an array of samples. In contrast, in the previous bonus exercise, ω was passed as a distribution, and we performed the sampling for ω in the simulation function.

Then, we complete the code in *assignment_1.py*. We begin by defining the parameters of our experiment in the *main* function, similarly to *Bonus Exercise 1*, such as the deterministic and the stochastic parameters of the oscillator, the bounds of

ω (relevant as upper and lower bounds for the interpolation grid), the time point of interest $t_{target} = 10$, the numbers of interpolation points $N = [2, 5, 10, 20]$ and numbers of Monte Carlo samples $M = [10, 100, 1000, 10000]$ for the different experiment runs. We would like to elaborate that we use exactly n interpolation points in our implementation, so for $n = 2$, we use 2 interpolation points. In the tutorials, it was mentioned that $N + 1$ is a possible convention for the number of points in some implementations; it depends on how exactly the task is formulated and on how N is defined. If needed, our code can be easily modified to fit with such an implementation.

To ensure reproducibility of our results, we set the seed to 42 for all random operations in the implementation. We conduct the experiment as follows:

- For each $n \in N$, we first **fit the Lagrange interpolator**. To do this, we need the values of the oscillator at t_{target} for each of the interpolation points. Therefore, with our helper function, we generate the Chebyshev interpolation grid with n nodes within the bounds of ω . We pass each of the n nodes of the grid as ω and discretize the oscillator up to t_{target} . The oscillator values at the time of interest are at the last index $[-1]$ of the return value of *discretize*. With those values and the interpolation grid, we instantiate and fit the Lagrange interpolator. We time how long it takes to complete this substep using the *time* library. As mentioned in the assignment, in this step we perform n function (oscillator) evaluations over the interpolation grid.
- For each $n \in N, m \in M$, we then **evaluate the Lagrange interpolant with Monte Carlo**. Since our interpolator is already fitted in the previous step, we only need to obtain m ω Monte Carlo samples for ω before evaluating the interpolator for each of them. We also time this substep.
- We save the results and the durations of the previous two substeps for each combination in two $|N| \times |M|$ matrices. Regarding timing, we try to calculate the measurements for the experiment so that the results are similar to what we would see if each combination of $N \times M$ were executed completely independently of the others. This is to say that even though we fit the Lagrange interpolator once per given $n \in N$ and then reuse the fitted interpolator in the evaluation upon Monte Carlo sampling for each $m \in M$, the full duration of the fit for the given n is counted towards the duration of each experiment combination for this particular n and all m .
- Then for each $m \in M$, we perform **forward propagation UQ with m Standard Monte Carlo** samples for ω , like in the previous bonus exercise, and time the duration of this step. We only simulate the oscillator up to the time of interest t_{target} and then save the oscillator values at t_{target} , which are at the last index $[-1]$ of the return array of the *simulate* function.
- The next step is to compute the relative errors of the values of interest, mean and variance, compared to the provided reference values. We perform this for all saved oscillator estimations across the different experiment runs.
- Largely adapting the code from *Bonus exercise 1*, we plot the errors on log-log scales and print the collected timing information.

1.2 Findings

The plotted relative errors of the mean and the variance for all experiment runs on a log-log comparison to the number of m Monte Carlo samples are shown in figures 1 and 2.

We are including the figures in as large a size as possible and have configured different dash sizes for the line styles for the different experiment runs of the Lagrange interpolation with MC. Upon zooming in, it is visible that the errors for forward propagation of UQ with Standard Monte Carlo and Lagrange interpolation with MC for $n \geq 5$ overlap. From the previous bonus exercise, we know that for forward propagation of UQ with Standard MC, the error decreases as the number of samples M increases for $M \geq 100$. Also, we have discussed that in the case of forward propagation of UQ and calculation of relative error instead of standard error, the relationship between convergence of the error and M is more complicated than $O(\sqrt{M})$ and between $m = 10$ and $m = 100$, the error even unexpectedly slightly increases. These findings now also apply to Lagrange interpolation with MC with at least $n \geq 5$ interpolation points. In addition, increasing the number of interpolation points N from 5 to 20 does not significantly affect the error.

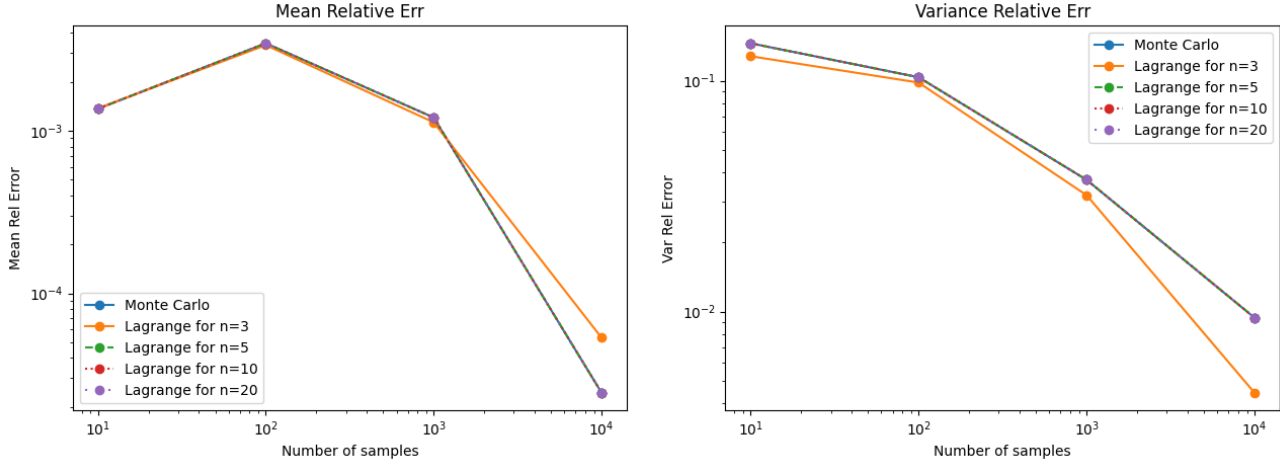


Figure 3: Relative errors of mean and variance, $N = [3, 5, 10, 20]$

We also find that 2 interpolation points produce the most inaccurate result, as expected, since interpolation over 2 points is linear and not an accurate oscillator approximation. In this perspective, having $N + 1$ interpolation points could have been a better interpretation for the number of interpolation points. Therefore, we re-ran our code with $N = [3, 5, 10, 20]$ as the input for the number of interpolation points. The resulting error plots are in figure 3 and show that for $n = 3$, the errors for Lagrange with MC also behave in a similar trend to those for $n \geq 5$.

The timing was performed on a personal laptop with 8 Intel Cores running at 2.5 GHz with dynamic frequency scaling enabled. All user programs, except the development environment, were closed. However, this is a multi-purpose machine, and after a few years of use and many additional installations, multiple backend processes such as Windows updates, proprietary software updates, antivirus scans, VPNs, etc., might impact the cache, trigger dynamic CPU frequency scaling, and affect the exact timing numbers. Also, the repetition of experiments over the same $n \in N$ and the same fitted interpolator weights across different $m \in M$ might have introduced additional cache speedups in the later experiment runs. Therefore, we interpret the timing results as trends rather than exact, fully reproducible values. For more reproducible timing, it would be better to use a dedicated virtual machine with a well-documented hardware-software stack.

Method	M=10	M=100	M=1000	M=10000
Monte Carlo	0.005535602569580078	0.05268239974975586	0.5294477939605713	5.323477745056152
Lagrange, N=2	0.0017473697662353516	0.0015332698822021484	0.0015826225280761719	0.0023217201232910156
Lagrange, N=5	0.0030248165130615234	0.0029783248901367188	0.003045797348022461	0.003842592239379883
Lagrange, N=10	0.005445718765258789	0.0054171085357666016	0.005509138107299805	0.006634712219238281
Lagrange, N=20	0.010693550109863281	0.01069331169128418	0.010808944702148438	0.013218164443969727

Table 1: Execution times (in seconds) for Monte Carlo, and Lagrange interpolation with Monte Carlo for sample sizes M and interpolation points N

Nonetheless, the timing results in table 1 clearly show that Lagrange interpolation with Monte Carlo sampling is faster than forward propagation UQ with Standard Monte Carlo for $M \geq 100$. The time consumed for forward propagation UQ with Standard Monte Carlo scales up almost linearly with M . For larger M , Monte Carlo estimation of the Lagrange interpolant is a faster approach. The logical reason is that the slowest operation in our code is the function (oscillator) evaluation. With Standard Monte Carlo, this operation is performed M times and for Lagrange interpolation with MC, it is performed only N times (during the fit substep).

From the error comparisons, we know that for $M = 10000$ and forward propagation UQ with Standard Monte Carlo, as well as Lagrange interpolation with MC for $N \geq 5$ have the least relative errors of the mean and the variance and are equally numerically correct oscillator estimators. Looking at the timing as an additional differentiator between these approaches,

Lagrange interpolation with MC for $N = 5$ (in blue in table 1) emerges as the fastest between the most accurate of our experiment runs. In terms of powers of 10, it is about 10^3 times faster than forward propagation UQ with Standard Monte Carlo.

Interestingly, there are small speedups in Lagrange interpolation with MC across the experiment runs for the same n for $m = 10$ and $m = 100$. This could have multiple reasons - caching of the interpolator weights, dynamically upscaled frequency due to a more computation-heavy process, better parallelism of operations over larger M , etc.

2 Assignment 2

2.1 Assignment 2.1 - Proof

We want to show that we can check whether two polynomials are orthogonal or orthonormal by computing the expected value $\mathbb{E}[\phi_i(X) \cdot \phi_j(X)]$.

Let X be a continuous random variable with an associated PDF $\rho(x)$. Then,

$$\mathbb{E}[u(X)] = \int u(x)\rho(x)dx$$

If we set $u(X) = \phi_i(X) \cdot \phi_j(X)$ in the above, we get:

$$\mathbb{E}[\phi_i(X) \cdot \phi_j(X)] = \int \phi_i(x)\phi_j(x)\rho(x)dx$$

As per the definition of the inner product of two polynomials from the assignment:

$$\langle \phi_i(x), \phi_j(x) \rangle_\rho = \int \phi_i(x)\phi_j(x)\rho(x)dx$$

Hence,

$$\boxed{\mathbb{E}[\phi_i(X) \cdot \phi_j(X)] = \langle \phi_i(x), \phi_j(x) \rangle_\rho}$$

and to check whether two polynomials are orthogonal or orthonormal, we can calculate $\mathbb{E}[\phi_i(X) \cdot \phi_j(X)]$ for a continuous random variable X . With the definitions from the assignment, it follows that if the expected value of the inner product is equal to $\gamma_i \delta_{ij}$ where $\gamma_i \in \mathbb{R}$ and δ_{ij} is the Kronecker delta function, the polynomials are orthogonal. If the expected value of the inner product is equal to δ_{ij} , they are orthonormal.

2.2 Assignment 2.2 - Implementation

For this assignment, we implement the above theory numerically.

We start by defining the parameters of our experiment. Then, we generate two sets of orthonormal polynomials of degree up to $N = 10$ with respect to two distributions $\rho_1 \sim \mathcal{U}(-1, 1)$ and $\rho_2 \sim \mathcal{N}(5, 1)$. For the generation, we use the *generate_expansion* function from the *Chaospy* library and set its parameter *normal = true* to get orthonormal instead of orthogonal expansions. We could normalize the orthogonal expansions ourselves but the calculations might introduce additional error.

Afterwards, we use the *E Chaospy* function to calculate the expected value of the inner products of the generated polynomials for both distributions. We save the results in two matrices. In case the polynomials are orthonormal as we generated them to be, $\mathbb{E}[\phi_i(X) \cdot \phi_j(X)] = \delta_{ij}$ with $\delta_{ij} = 1, i = j$ and $\delta_{ij} = 0, i \neq j$. Therefore, we expect that the matrices should look like the identity matrix I . However, some values may deviate slightly due to computational errors in the generation and/or in the inner product check.

To check the results, we subtract I from both matrices and present the deviations as diverging color maps that are discussed in the next subsection. For perfectly orthonormal polynomials and no computational error, both resulting deviation matrices should be zero matrices.

2.3 Assignment 2.2 - Results

Figure 4 presents the pink to green color maps for the differences between the calculated matrices of the expected value of the inner product of the generated polynomials up to $N = 10$ with respect to $\rho_1 \sim \mathcal{U}(-1, 1)$ and $\rho_2 \sim \mathcal{N}(5, 1)$. We use a diverging color map in which values near 0 look white. Therefore, we expect that, both color maps should be white. However, we observe deviations, mostly at higher N , and more so for the normal distribution than for the uniform one. Also, interestingly, the color maps are not symmetrical matrices, so that means that the deviations for $\mathbb{E}[\phi_i(X) \cdot \phi_j(X)]$ and $\mathbb{E}[\phi_j(X) \cdot \phi_i(X)]$ are different.

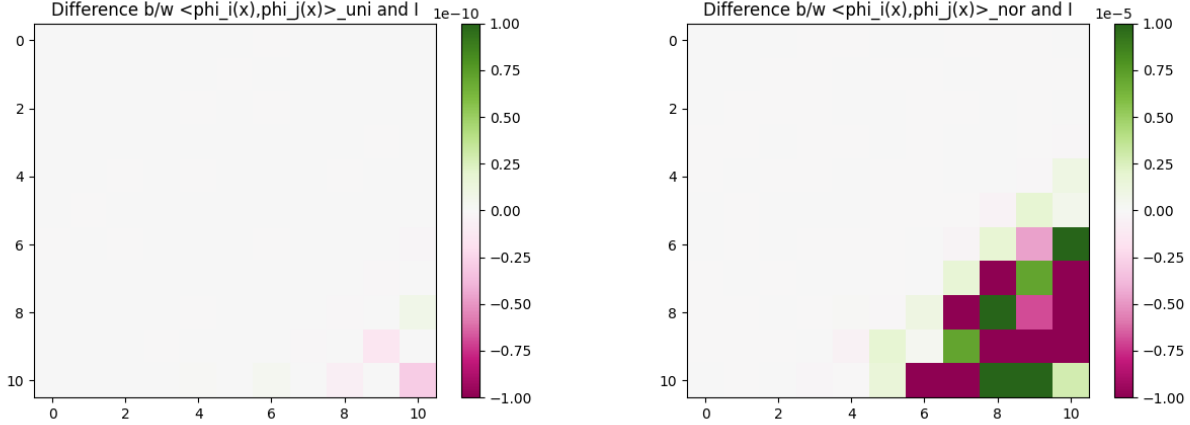


Figure 4: Differences between obtained inner products and the identity matrix as color maps

While experimenting with the code, we printed the matrices and saw that the highest absolute deviation for the uniform distribution polynomials is about $2.91e-11$. Hence, we configured the diverging color map for the uniform distribution with the interval $[-1e-10, 1e-10]$. This scale was too granular for the deviation of the normal distribution polynomials, where the highest absolute error is about $1.29e-3$. Therefore, we opted to configure the diverging color map for the normal distribution with a larger interval $[-1e-5, 1e-5]$.

In the printed matrices, we could also see that the error is 0 only for N up to 2 for the normal distribution, whereas even the squares for $i=10, j=9$ have an error of 0 for the uniform distribution. Nevertheless, no error exceeds $1.3e-3$, which is already quite small. Therefore, such generated orthonormal polynomials and the check for orthonormality through expected value might be useful in numerical applications, given that a slight deviation is tolerable.

3 Assignment 3

3.1 Proof 1

We want to show that, for fixed t , $\mathbb{E}[f^N(t, \omega)] = \hat{f}_0(t)$ where $f^N(t, \omega) = \sum_{i=0}^{N-1} \hat{f}_i(t) \phi_i(\omega)$.

Since t is fixed, the coefficients $\hat{f}_i(t)$ do not depend on the random variable ω . Therefore, using linearity of expectation,

$$\mathbb{E}[f^N(t, \omega)] = \mathbb{E}\left[\sum_{i=0}^{N-1} \hat{f}_i(t) \phi_i(\omega)\right] = \sum_{i=0}^{N-1} \hat{f}_i(t) \mathbb{E}[\phi_i(\omega)].$$

Now we use that expectation with respect to the density ρ is given by integration against ρ :

$$\mathbb{E}[\phi_i(\omega)] = \int_{\text{supp}(\rho)} \phi_i(\omega) \rho(\omega) d\omega.$$

Since $\phi_0(\omega) = 1$, this can be written as

$$\mathbb{E}[\phi_i(\omega)] = \int_{\text{supp}(\rho)} \phi_i(\omega)\phi_0(\omega)\rho(\omega) d\omega.$$

By orthonormality of the basis functions,

$$\int_{\text{supp}(\rho)} \phi_i(\omega)\phi_0(\omega)\rho(\omega) d\omega = \delta_{i0}.$$

Therefore

$$\mathbb{E}[\phi_i(\omega)] = \delta_{i0}.$$

Hence

$$\mathbb{E}[f^N(t, \omega)] = \sum_{i=0}^{N-1} \hat{f}_i(t)\delta_{i0}.$$

Since $\delta_{i0} = 1$ only for $i = 0$ and is 0 otherwise, only the first term remains:

$$\mathbb{E}[f^N(t, \omega)] = \hat{f}_0(t).$$

Thus,

$$\boxed{\mathbb{E}[f^N(t, \omega)] = \hat{f}_0(t)}$$

3.2 Proof 2

We want to show that, for fixed t ,

$$\text{Var}[f^N(t, \omega)] = \sum_{i=1}^{N-1} \hat{f}_i^2(t).$$

Using the identity

$$\text{Var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2,$$

with $X = f^N(t, \omega)$, we obtain

$$\text{Var}[f^N(t, \omega)] = \mathbb{E}[(f^N(t, \omega))^2] - \mathbb{E}[f^N(t, \omega)]^2.$$

From Proof 1,

$$\mathbb{E}[f^N(t, \omega)] = \hat{f}_0(t).$$

It remains to compute the second moment. Since

$$f^N(t, \omega) = \sum_{i=0}^{N-1} \hat{f}_i(t)\phi_i(\omega),$$

we have

$$\mathbb{E}[(f^N(t, \omega))^2] = \mathbb{E}\left[\left(\sum_{i=0}^{N-1} \hat{f}_i(t)\phi_i(\omega)\right)\left(\sum_{j=0}^{N-1} \hat{f}_j(t)\phi_j(\omega)\right)\right].$$

Expanding the product and using linearity of expectation gives

$$\mathbb{E}[(f^N(t, \omega))^2] = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \hat{f}_i(t)\hat{f}_j(t)\mathbb{E}[\phi_i(\omega)\phi_j(\omega)].$$

By orthonormality,

$$\mathbb{E}[\phi_i(\omega)\phi_j(\omega)] = \delta_{ij}.$$

Therefore,

$$\mathbb{E} \left[\left(f^N(t, \omega) \right)^2 \right] = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \hat{f}_i(t) \hat{f}_j(t) \delta_{ij} = \sum_{i=0}^{N-1} \hat{f}_i^2(t).$$

Hence

$$\text{Var} \left[f^N(t, \omega) \right] = \sum_{i=0}^{N-1} \hat{f}_i^2(t) - \hat{f}_0^2(t) = \sum_{i=1}^{N-1} \hat{f}_i^2(t).$$

Thus,

$$\boxed{\text{Var} \left[f^N(t, \omega) \right] = \sum_{i=1}^{N-1} \hat{f}_i^2(t)}$$

4 Assignment 4

4.1 Implementation

4.1.1 Computing coefficients

Both the manual and chaospy method use the following formula under the hood:

$$\hat{f}_i(t) = \int_{\text{supp}(\rho)} f(t, \omega) \phi_i(\omega) \rho(\omega) d\omega \approx \sum_{k=0}^{K-1} f(t, x_k) \phi_i(x_k) w_k \quad (1)$$

Hence in both cases we need to compute the terms $f(t, x_k)$, which we do by:

1. Setting up a time grid for our desired interval $t \in [0, 10]$
2. Assuming that we have the nodes, generated using quadrature, we take them as the samples for omega
3. Using the model parameters and initial condition of the oscillator we simulate $f(t, x_k)$. We take this as our model evaluation to approximate or also called the `solves` parameter.

Manual method

For the manual method we use direct translation of the formula, i.e. for k from 0 to $K - 1$ we take the sum of the respective simulated solution, polynomial applied to the variable and weight.

chaospy method

For the `chaospy` approach we need to use the `cp.fit_quadrature` method on the polynomials, nodes, weights and `solves`. It's important to set `retall=1`, which sets the output to be polynomials and Fourier coefficients (i.e. coefficient of orthogonal expansion).

4.1.2 Computing moments

As proven in Assignment 3, for the mean and variance we take respectively the 0th coefficient and the sum of the coefficient squares starting from index 1. That's computed manually with indexing and `numpy`.

4.1.3 Benchmarking and relative error

The goal is to compute the relative error of the pseudo-spectral approach mean and variance with respect to the reference Monte Carlo values. In the meantime we also compare whether the manual and `chaospy` methods produce different results. To execute this computation we fulfill the following steps:

1. Defining the simulation variables $t \in [0, 10]$, model parameters $c = 0.5$, $k = 2.0$, $f = 0.5$, initial condition $y_0 = 0.5$, $y_1 = 0.0$ and distribution $\omega \sim \mathcal{U}(0.95, 1.05)$

2. Then for every $K = N = [1, 2, 3, 4, 5, 6]$:
 - (a) We generate the nodes and weights using `cp.generate_quadrature` for $K - 1$. We use the Gaussian rule due to uniformly distributed stochastic variable.
 - (b) We then compute the polynomials $\phi_i(\omega)$ using `cp.generate_expansion` for $N - 1$. We need to set `normed=True` to ensure they're orthonormal
 - (c) Finally we compute the coefficients with the manual and `chaospy` approach
3. Using the coefficients we compute the mean and variance and their relative error to the MC values using the formula:

$$relative_error = \frac{|Pseudospectral_value - MC_value|}{|MC_value|}$$

4.2 Observations

In Table 2, we can see that the coefficients generated using the manual and `chaospy` approach are nearly identical. The difference could be attributed to numerical errors, e.g. rounding.

Table 2: Manual and Chaospy coefficients and their absolute difference for $N = K = 6$.

i	Manual	Chaospy	Difference
0	$-4.3896850457 \times 10^{-1}$	$-4.3896850457 \times 10^{-1}$	$1.1102230246 \times 10^{-16}$
1	$-6.7734656242 \times 10^{-3}$	$-6.7734656242 \times 10^{-3}$	$1.4745149546 \times 10^{-17}$
2	$1.2265227548 \times 10^{-2}$	$1.2265227548 \times 10^{-2}$	$1.3287981826 \times 10^{-15}$
3	$1.8731625890 \times 10^{-4}$	$1.8731625890 \times 10^{-4}$	$1.9046721665 \times 10^{-16}$
4	$-4.8231747760 \times 10^{-5}$	$-4.8231747761 \times 10^{-5}$	$4.7732000644 \times 10^{-16}$
5	$-5.4135548981 \times 10^{-7}$	$-5.4135550990 \times 10^{-7}$	$2.0092676065 \times 10^{-14}$

As the polynomials, nodes and weights are shared when computing the moments (with only the coefficients differing by approach), the mean and variance relative errors are visually identical between the manual and `chaospy` approach (See Figure 5). This is expected as both approaches utilize formula (1) under the hood.

Looking closer at the *mean error* subplot on Figure 5, we can see that it starts of high, drops significantly by $N = K = 2$, further slightly oscillates until $N = K = 4$ and finally stagnates. This means that the mean converges at $N = K = 4$ to the true, giving us the best approximation with the least amount of computation. The residual floors are the sampling error of MC ($\hat{f}_0 - E_{\text{ref}} = -0.4389685 - (-0.43893703) = 7.2 \times 10^{-5}$).

When observing the *variance error* subplot on Figure 5, we see that it starts at 1, decreases significantly both for $N = K = 2$ and $N = K = 3$, further refines until $N = K = 5$ and stays the same for point 6. The initial value of 1 is because when computing the variance, we skip over the first coefficient, meaning for $N = K = 1$ we have variance 0. Afterward, the pseudo-spectral variance converges at $N = K = 5$, giving us the most accurate estimation. The residual floors again represent the sampling error of MC. In summary, for $N = K = 5$ we have the best approximation with the smallest error for both the mean and variance.

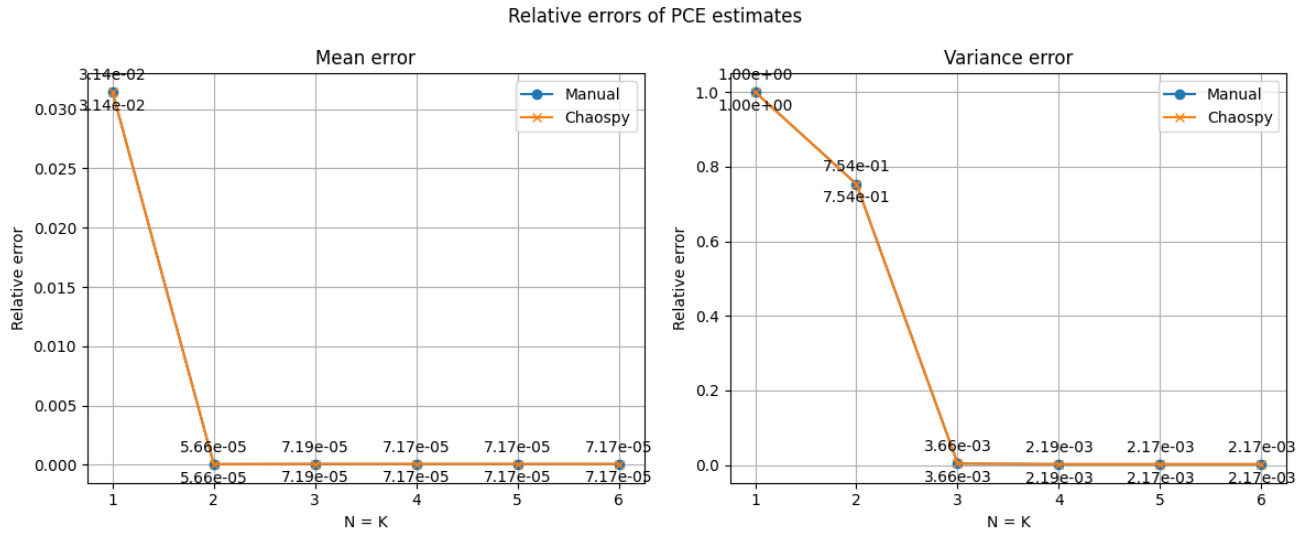


Figure 5: Relative errors of the mean (left) and variance (right) with respect to the Monte Carlo references values